ADR 0002: RDM reads the spec as Gherkin and audits it with Allure

Document ID	adr-0002-rdm-reads-gherkin-and-allure
Revision	de6c942
Date	2026-11-10
Status	Controlled in git

Contents

1 Context 3
2 Decision 3
3 Consequences 4
4 Evidence is Allure only 4

> Status: Accepted (2026-11-10). It replaces RDM's earlier model, in which RDM held the requirements itself as design inputs.

1 Context

RDM used to own the specification. A project wrote **design inputs** (`DI-n` in the frontmatter of one design document per bounded context), traced each to a user need registered in the V&V plan, tagged one test per input with `@allure.story("DI-n")`, and RDM's design gate, scaffolding command and git hooks policed how that record was written.

Software is now increasingly specified and regenerated elsewhere: an agentic factory such as Löyly keeps the specification as stamped Gherkin scenarios, with Allium rules, an event model and an event storm behind them, and rebuilds the code from it. Keeping the same requirement a second time as a design input in RDM makes two hand-kept copies of one fact, which drift.

What a regulated release needs from RDM is not authorship but audit: is every clause of the applicable standards addressed by a document, is every requirement verified by a passing test at the released commit, is every risk controlled, and who approved it.

2 Decision

RDM reads a spec; it never writes it. Its contract with a spec is Gherkin and Allure, and nothing else is fixed.

- **Requirement.** A Gherkin scenario tagged with an id is a requirement. By default any id-shaped tag (`@REQ-12`, `@item-02-001`) is an id; a spec can name its own pattern. A scenario without one is reported, and blocks a release; so does an id on two scenarios. Feature files are parsed with Cucumber's own parser (`gherkin-official`).
- **Evidence.** A test run claims the requirements it verifies with Allure `tag` labels naming their ids: what a Gherkin runner writes from a scenario's tags, or `@allure.tag("<id>")` on a plain test.
- **User need.** By default each Gherkin Feature, its description the need's text, addressed by the scenarios under it.
- **Everything else is configuration or an adapter.** `spec:` in the DHF's `config.yml` names the feature files (globs, exclusions), the id pattern and the needs source; without it RDM looks for `spec/` or `features/` at the repository root. A layout adapter reads one factory's richer spec: Löyly's (`rdm/specification/loyly.py`) reads its plan of slices and takes its user needs from the event model's screens, their Allium surfaces and the event storm.
- **What RDM keeps.** Gap analysis of documents against compliance checklists; traceability from Allure runs to requirements and user needs; the release gate over requirements, needs, risks and validation records; the knowledge graph and its MCP server; the regulatory documents, the verification report and the evidence bundle.
- **What RDM no longer has.** Design inputs and the V&V plan's user-need registry; the design gate, its git hooks and `rdm story new-input`; source scanning of test tags; and RDM's own

process ceremony (design reviews, the agent workflow), which the spec's owner covers in its own process.

3 Consequences

- One copy of each requirement: changing a scenario changes what the release gate counts, with no second edit.
- RDM works with any Gherkin spec and any Allure-writing test runner; a factory's richer model is one adapter, not RDM's model.
- With Features as needs, a scenario addresses exactly one need, the Feature it sits under; a spec that needs a scenario to serve several needs uses an adapter (Löyly's covers them by the Allium rules and guarantees it tags).
- RDM's own record follows the same model: its spec is written the Löyly way in `.pi/` (stamped Given/When/Then scenarios, Allium surfaces, event model, event storm), which `rdm init` and `rdm adopt` also lay down for a new spec; its tests claim requirements with `@allure.tag("item-NN-SSS")`, and its risk controls name requirement ids. Plain Gherkin stays supported: the contract is still only Gherkin and Allure. Against Löyly's `spec-v2`, the adapter reads 201 requirements, Löyly's own checklist.
- Breaking for projects on the old model: their design inputs become tagged scenarios, their user needs Features, their story tags Allure `tag` labels.

4 Evidence is Allure only

Graded verdicts (Löyly's sealed graders) are not evidence in RDM: they stay the spec owner's own check. A test run that verifies a requirement writes Allure results, and those alone count. A factory that wants a graded verdict counted turns it into an Allure result tagged with the requirement's id.